

AGENTIC MIGRATION PLATFORM

Migration legacy gouvernée,
traçable et vérifiable.

Une Migration Factory gouvernée pour
Java / Spring Boot et Angular

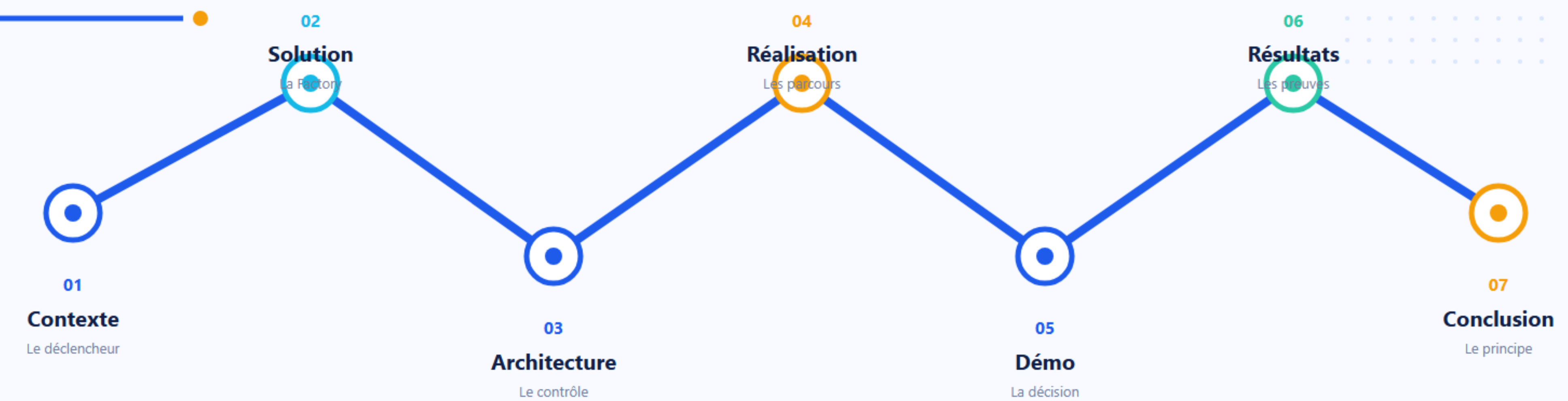
● PROPOSER → EXÉCUTER → PROUVER

Système de migration · humains, agents, services déterministes, preuves



Le chemin de la soutenance

Du problème opérationnel à la preuve d'une automatisation gouvernée



Un parcours qui relie intention → action → validation → preuve

Le déclencheur : migrer sans interrompre

Un portefeuille legacy doit évoluer vers le cloud à grande échelle

2027

cible cloud

Un grand client aérien

doit faire évoluer un parc legacy vers le cloud — sans perdre la continuité.

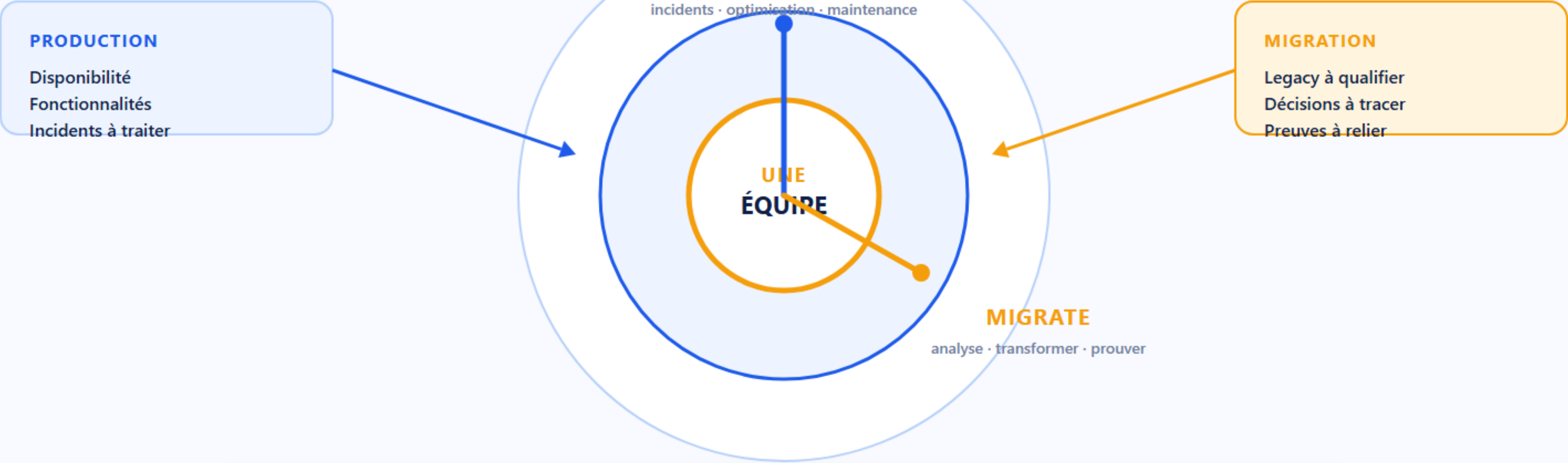
LA QUESTION

Comment migrer à l'échelle sans perdre le contrôle ?



Une équipe. Deux horloges.

Maintenir la production et avancer la migration en parallèle

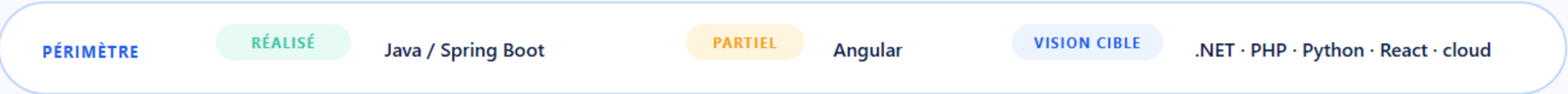
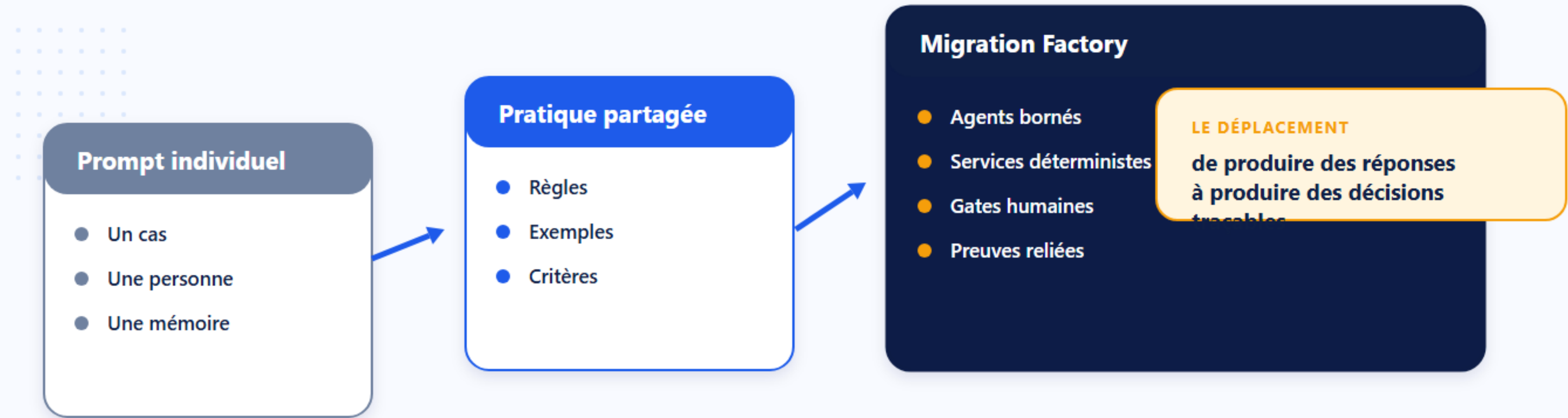


Quand le contexte est dispersé, le rework devient la troisième horloge.

perte de contexte décisions dispersées preuves difficiles à relier

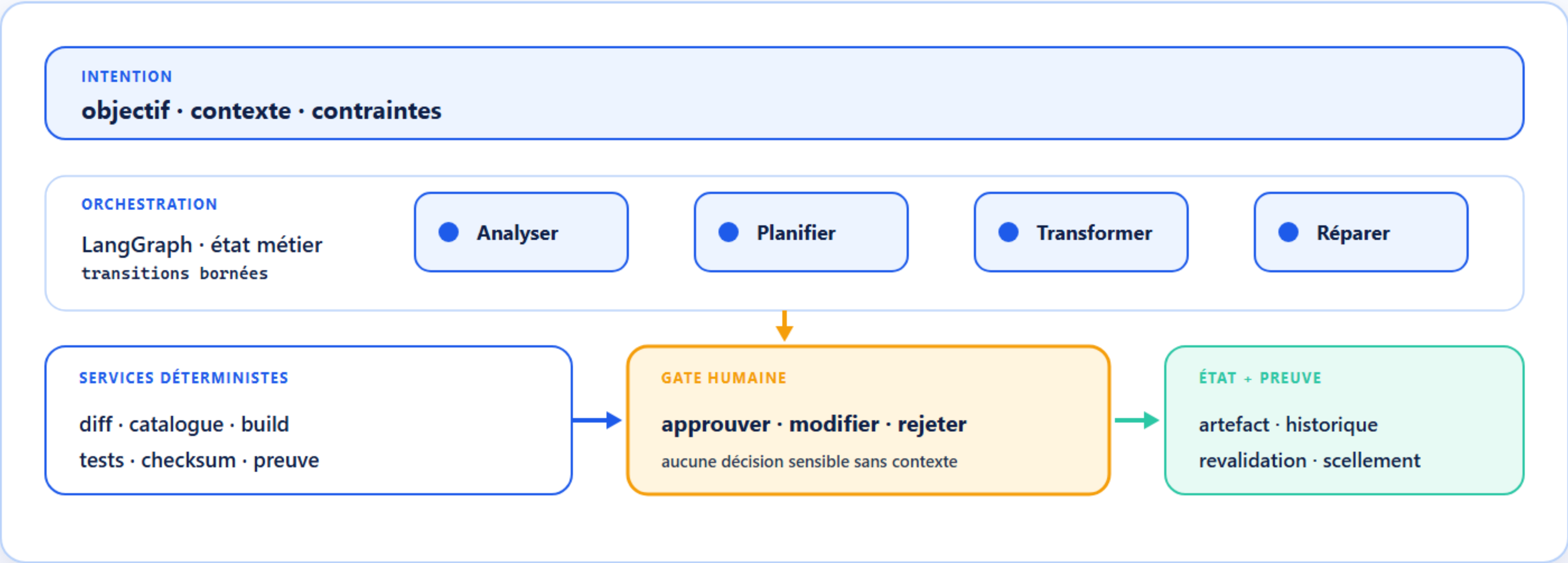
Du prompt isolé à la Migration Factory

Capitaliser l'expertise ne suffit pas : il faut gouverner le parcours



Une architecture qui sépare les responsabilités

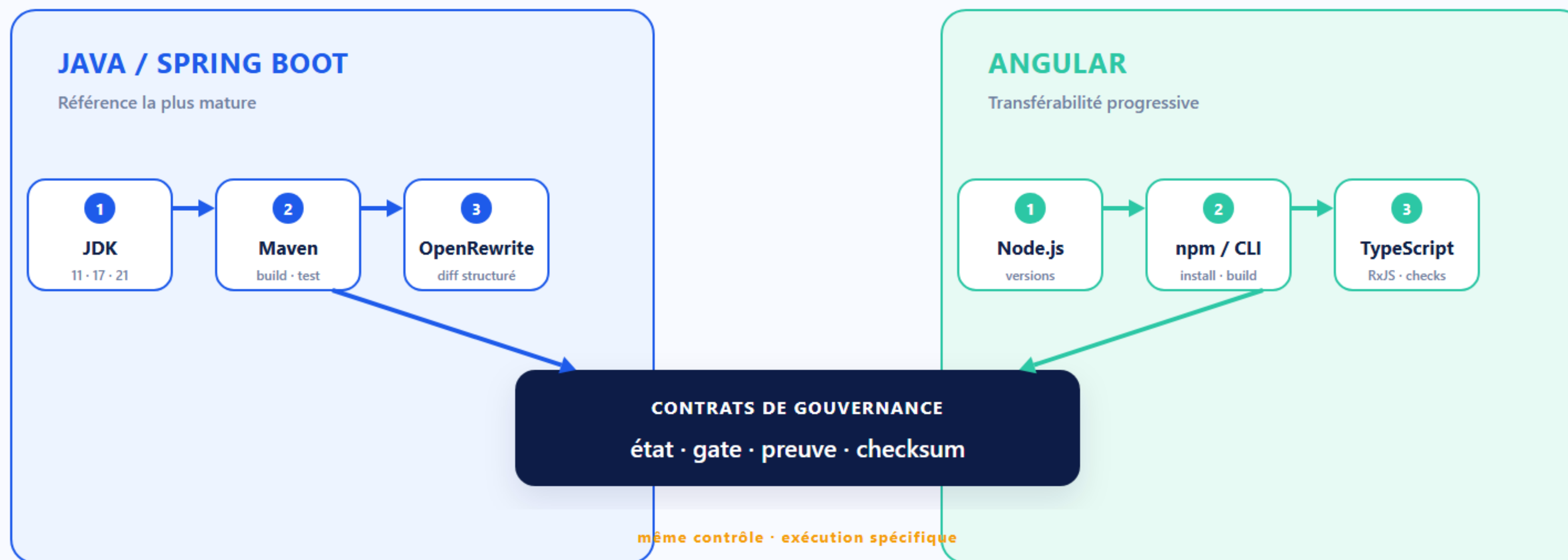
Les agents proposent. Les services exécutent. L'humain autorise.



Le système garde l'autorité là où le risque existe.

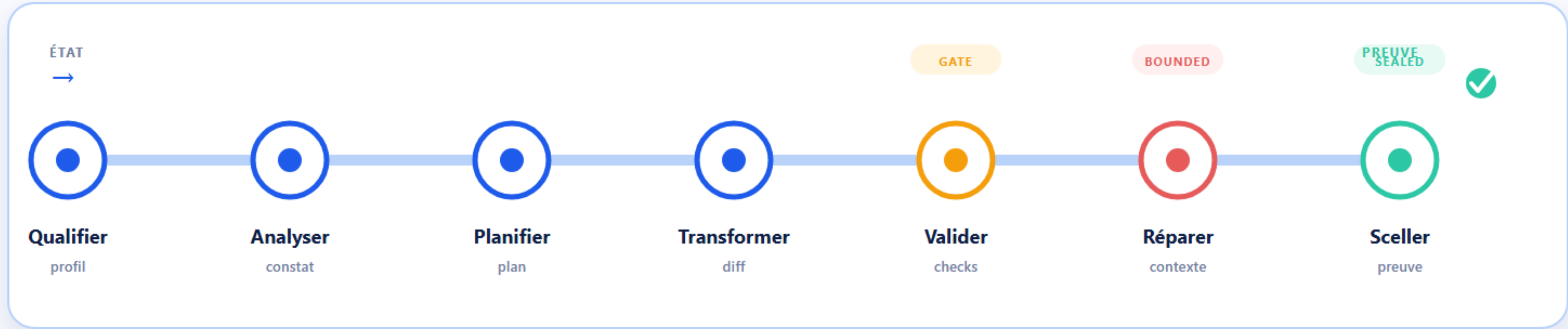
Deux toolchains. Un contrat de gouvernance.

Java et Angular restent distincts, mais parlent le même langage de contrôle



Le workflow complet : avancer par preuves

Chaque transition critique laisse un état, un gate et une trace



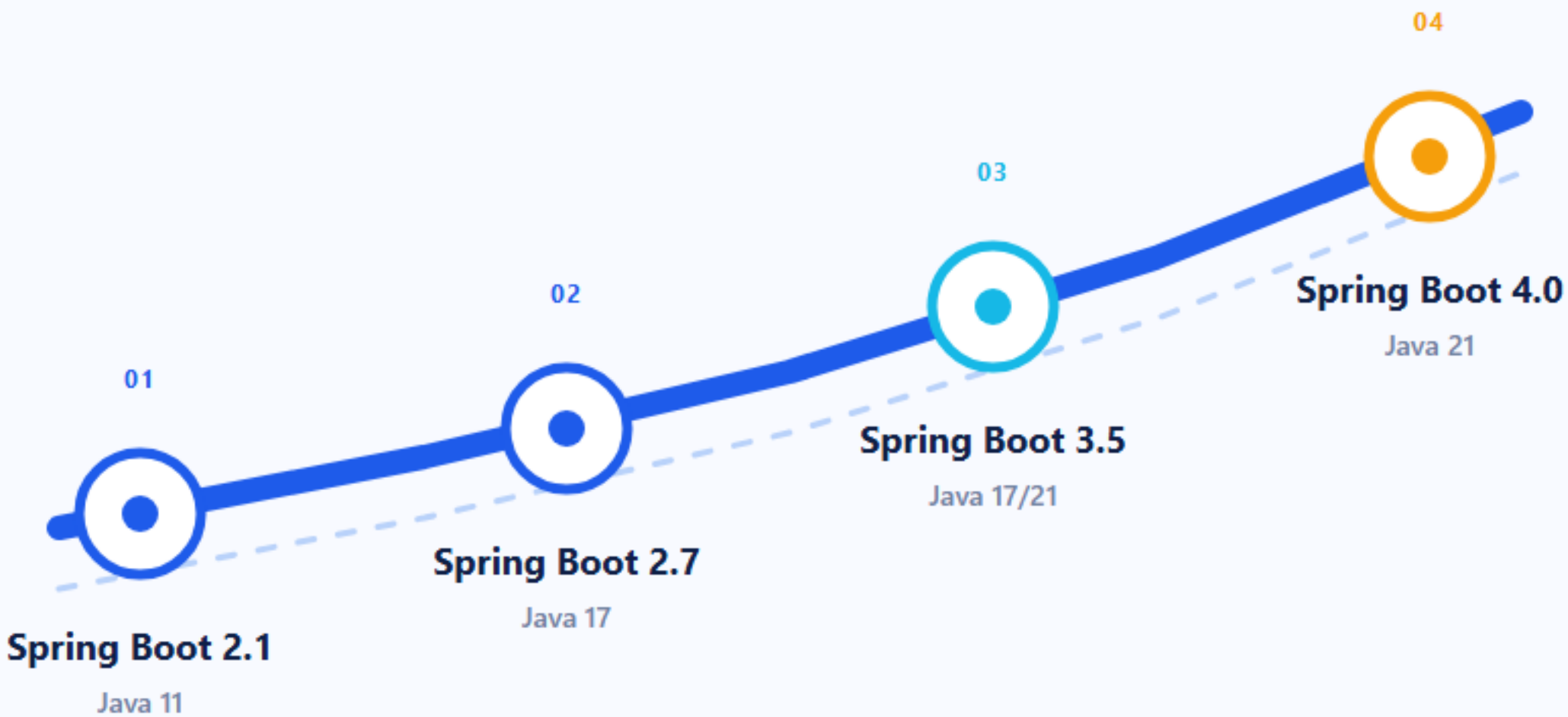
Aucune étape ne saute directement de l'intention à la production.

La progression se fait par artefacts, décisions et revalidation.

Axe 1 — Java / Spring Boot

Le parcours de référence le plus mature

UPGRADE RUNWAY



RÉFÉRENCE

Java / Spring Boot

trajectoire structurée
toolchain explicite
preuves revalidables

Maven

OpenRewrite

Java 11 · 17 · 21

Réalisation / partiel selon scénario

Axe 2 — Angular

Le modèle est transférable, la couverture reste progressive

PROOF STAIRCASE



FRONTIÈRE HONNÊTE

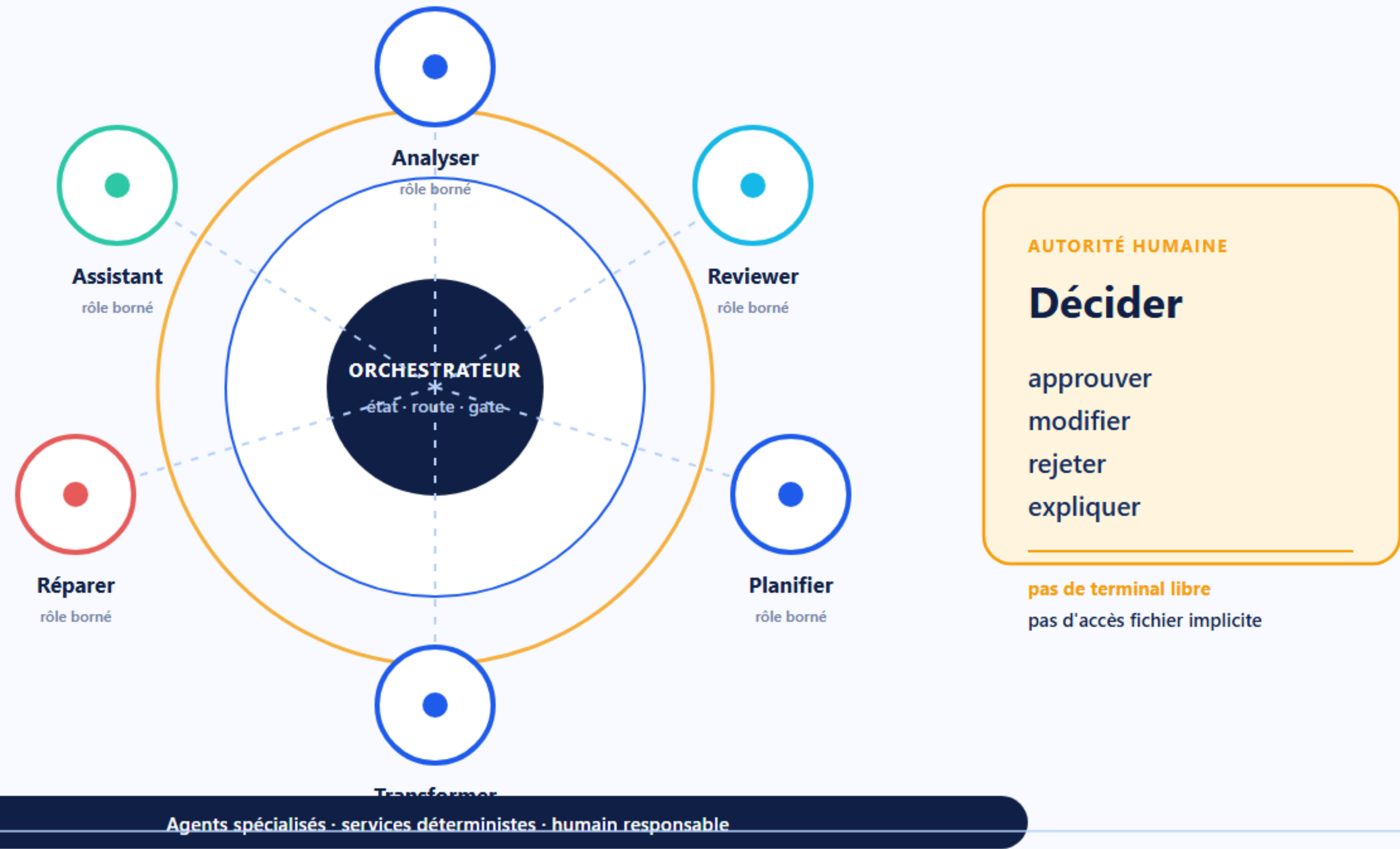
Pas de 11 → 21 réalisé.

18 → 19 scellée
19 → 20 scellée
20 → 21 préparée
non démarrée

Couverture partielle
présentée comme telle

Les agents proposent. Le système garde la frontière.

Spécialisation autour d'un orchestrateur, autorité finale côté humain



Réparer sans perdre la preuve

Un échec est gelé, contextualisé, revu, puis revalidé



CHECKSUM
même artefact · même état

PREUVE
build · test · trace

TENTATIVE BORNÉE
pas de retry aveugle

Démo — scénario utilisateur

Inspecter la proposition, vérifier la preuve, décider

SCÉNARIO LOCAL

G10

réparation Angular

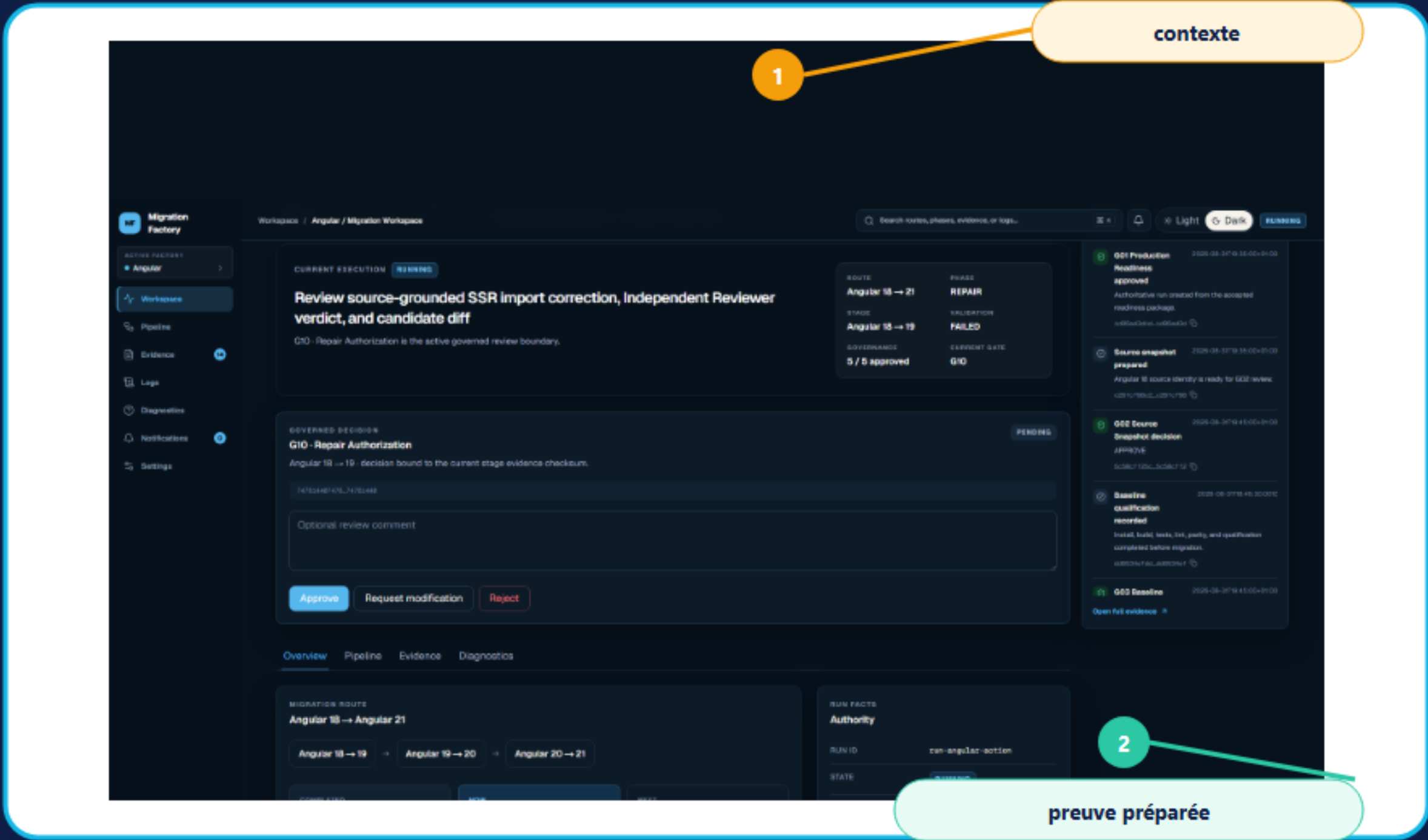
- 1

Inspecter
le contexte
- 2

Vérifier
la proposition
- 3

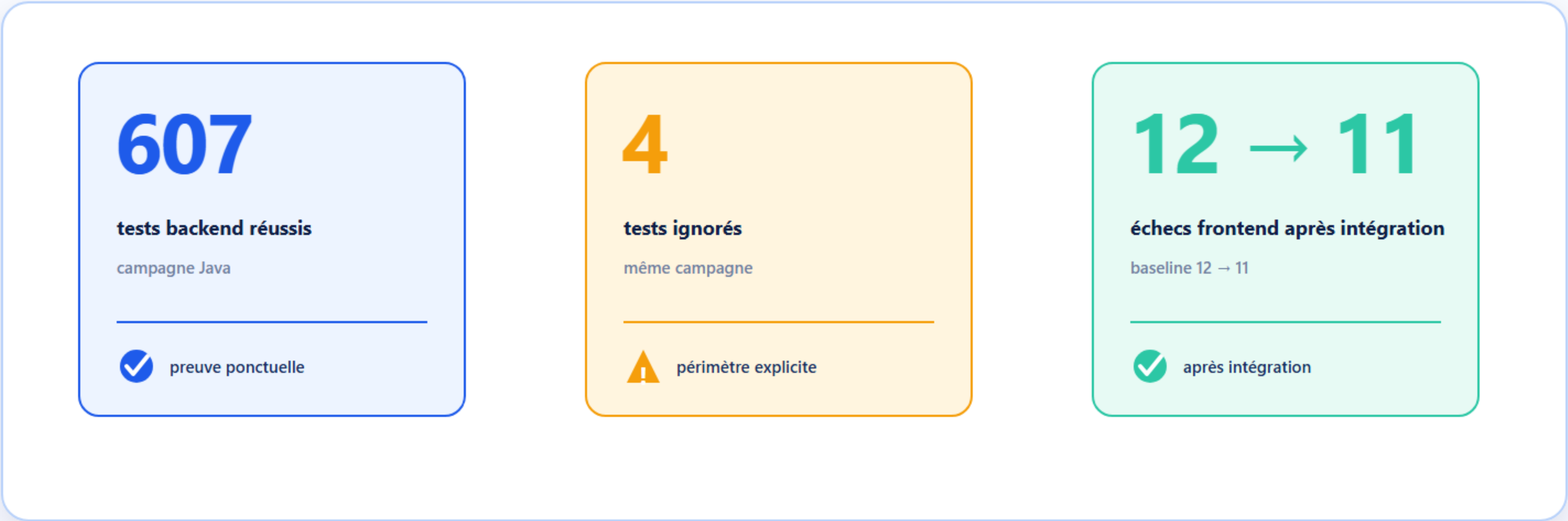
Décider
la suite

Démo locale — scénario anonymisé, états préparés



Les preuves montrent le contrôle

Traçabilité, validation et revalidation — pas un benchmark global



18 → 19 scellée · 19 → 20 scellée · 20 → 21 préparée

Des difficultés aux invariants

Chaque incident a renforcé un contrat d'architecture

CONTRAINTE

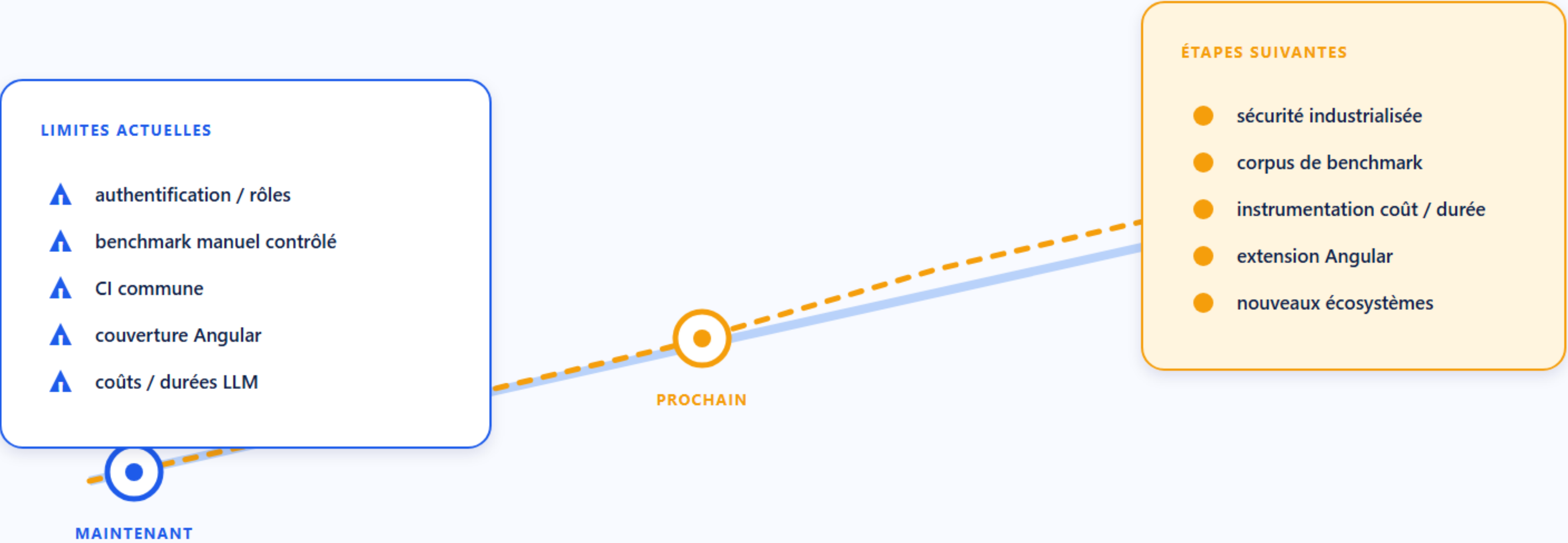
INVARIANT

!	Patch non applicable	→	✓	diff structuré + validation
!	Autorité de commande ambiguë	→	✓	manifeste backend
!	Profil Angular incompatible	→	✓	catalogue versionné
!	Incohérence SQLite	→	✓	base autoritaire réconciliée

La robustesse vient des frontières explicites, pas d'une promesse d'autonomie.

Le prototype est gouverné. L'industrialisation reste à construire.

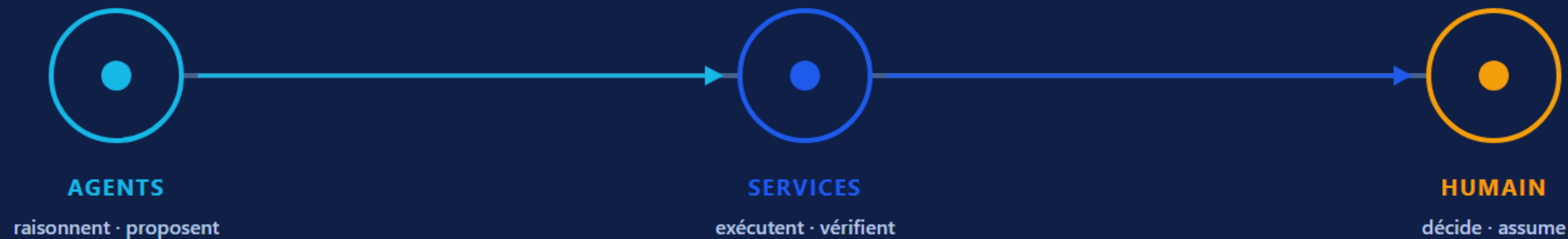
Séparer ce qui est livré de ce qui doit encore être mesuré ou sécurisé

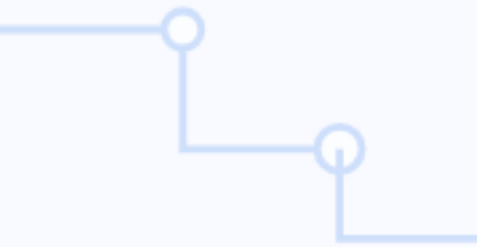


Les limites ne sont pas un échec : elles rendent la prochaine décision lisible.

Une migration agentique n'est pas une autonomie totale.

C'est un chemin où chaque décision reste gouvernée, traçable et revalidable.





Questions / Discussion



Architecture · preuves · limites assumées



Merci